

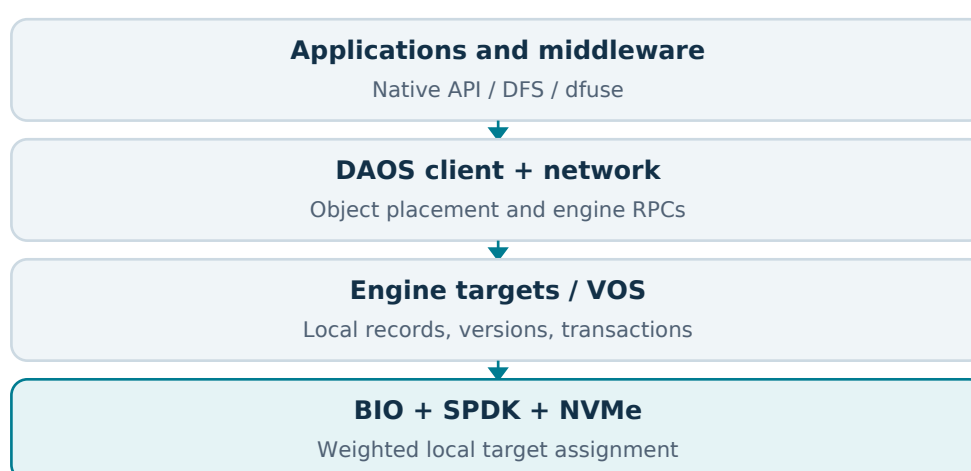
DAOS and heterogeneous NVMe

An implementation guide to static, per-engine weighted target assignment.

Version: 20 September 2026. Source baseline: DAOS 2.9.100, commit d0f733f6e. Status: experimental. This document describes this branch, not an upstream DAOS guarantee.

1. What DAOS does

DAOS is a distributed object store. Applications use its native API or an adapter such as DFS, dfuse, MPI-IO, or HDF5. Storage nodes run a management server and storage engines. Each engine exposes multiple targets, which provide parallel execution and storage resources. The management path provisions and monitors the system; application data flows through the client library and engine data path. [1]



VOS, the Versioning Object Store, manages local object records and versions. BIO, the Blob I/O layer, connects that local storage to SPDK devices and blobs. SMD, server metadata, persists the device-to-target and pool-blob mappings needed to reopen storage. Metadata placement depends on the configured storage mode; this feature does not require every deployment to have physical persistent-memory DIMMs. [3, 4]

The change in this branch: give faster devices more newly assigned targets inside one engine, using explicit relative weights. It changes BIO's device choice during assignment. It does not change the network protocol, object format, replication rules, or cluster-wide object placement.

2. From an application object to an SSD

A **pool** groups provisioned storage resources and tracks participating targets through a versioned pool map. A **container** groups objects and their properties. An **object class** describes the object's layout, including replication or erasure coding where configured. [2]

DAOS separates two placement decisions:

Decision	Input	Result	Changed here?
Distributed object placement	Object layout, pool map, fault domains	Shards on engine targets	No
Local device assignment	Target, storage role, SMD, device list	Device serving a target role	Yes, for new mappings

For an NVMe-backed operation, the conceptual path is:

1. The client resolves the object's layout and communicates with the appropriate engine targets. Jump placement traverses the pool's fault-domain hierarchy and handles shard separation and fallback choices. [5] 2. Target-local VOS code processes records, versions, indexes, and transactions. The configured object class and transaction rules determine which operations must complete. [3] 3. BIO uses the target's device/blob context to submit asynchronous storage I/O through SPDK. SPDK supplies block-device, blobstore, and I/O-channel abstractions. [4] 4. Completion propagates back through the engine and client stack. The weighting option does not weaken acknowledgement or data-protection requirements.

A target is not a drive. Several targets can share one drive. With separate storage roles, one target can use different devices for data, metadata, and WAL (write-ahead log). The implementation therefore checks role eligibility before comparing devices.

At startup, `assign_xs_bdev()` first consults SMD. If a mapping already exists, BIO uses that mapping. Only a missing mapping reaches `choose_device()`. This ordering is why changing a weight does not move existing data. [6]

3. Why equal assignment can underuse a fast device

Before this change, BIO selected the eligible device with the smallest mapped-target count. With equal workloads per target, that sends similar work to slow and fast devices. A faster device may then have spare bandwidth while a slower one limits throughput.

The new policy selects the eligible device with the smallest:

`mapped_target_count / configured_device_weight`

It compares ratios without floating point:

`candidate_count * best_weight < best_count * candidate_weight`

The products use 64-bit integers. Weights range from 1 to 65535. Ties preserve device-list order. An unspecified device has weight 1. With all weights equal and sufficient mapping slots, the previous assignment order is preserved. [6, 7]

Fresh deployment	Weights in device order	Targets	Result in device order
Two equal devices	1 : 1	6	3 : 3
Second device slower	2 : 1	6	4 : 2
Third device faster	1 : 1 : 2	8	2 : 2 : 4
Ratio cannot divide exactly	2 : 1	8	5 : 3

These examples assume equivalent storage roles and enough SMD slots. Assignment is incremental, so existing counts and device order affect rounding. The policy is not an exact global optimizer. At zero counts, the first eligible device wins a tie regardless of its weight; small target populations can poorly approximate extreme ratios.

Weights should reflect measured sustainable bandwidth or IOPS for the intended workload, not PCIe generation alone. More targets can increase parallel work reaching a fast device when traffic is spread across those targets. A single hot target remains tied to its assigned device. CPU, network, object layout, client concurrency, and slow replicas can still dominate.

4. What changed in the code

Source	Responsibility
src/bio/bio_weight.h	Strict parser, integer ratio comparison, mapping-slot guard
src/bio/bio_internal.h	In-memory weight on each BIO device
src/bio/bio_xstream.c	Startup validation, device weight loading, weighted selection
src/include/daos_srv/smd.h	Exposes the existing persistent 64-entry limit
BIO/SMD tests and test manifests	Policy tests, SMD boundary regression, CI and packaging

Configuration validation. Before opening or creating initial blobstores, BIO parses the environment setting and verifies that each named device exists in the selected SPDK bdev class. Empty values, duplicates, whitespace, malformed weights, and unknown names fail startup. Device creation reads the configured weight outside the I/O path. [6, 7]

Selection and persistence. BIO filters for the requested role, checks available SMD slots, and compares eligible devices by weighted count. `assign_roles()` then records the mappings. Existing target mappings still bypass selection. No persistent format or placement-layout version is changed.

The review fix: mapping capacity. SMD has 64 entries per device. Each role mapping consumes an entry, even when multiple entries have the same target ID. System-target mappings also consume entries although the runtime load counter deliberately excludes their increments. The selector now reads actual SMD occupancy and checks room for every role that `assign_roles()` will add before choosing a device. A full device is skipped even if its weight is very large. [6, 8]

For example, 21 targets using combined data/meta/WAL roles consume 63 entries. Another three-role assignment cannot fit. A device with room for all three mappings may be chosen instead. If no eligible device fits, assignment fails explicitly. The guard prevents this capacity error partway through one target's role assignment; it is not a transactional preflight or rollback for the entire engine initialization.

Build changes are separate commits. One permits libfabric to auto-detect optional PSM2/OPX dependencies. Another repairs copying SPDK runtime resources when the dependency prefix differs from the installation prefix. Neither is part of the weighting algorithm. The new test executable is included in the RPM server-tests manifest and the BIO unit-test runner.

5. Configuration and operational limits

Add the setting to the appropriate engine's existing server YAML:

```
engines:
  - targets: 8
    env_vars:
      - "DAOS_NVME_DEVICE_WEIGHTS=Nvme_A=1,Nvme_B=1,Nvme_C=2"
      # Keep the deployment's other engine settings.
```

The names are placeholders. Use the actual SPDK bdev names from that engine's generated configuration and logs. They are not Linux `/dev/nvme*` paths, PCI addresses, or device UUIDs. Existing engine environment forwarding supplies the setting to BIO; there is no new typed control-plane configuration field.

Lifecycle. Start with separately provisioned, disposable storage to compare fresh allocations. Restarting with different weights preserves SMD mappings. Creating another pool on already mapped targets also retains the device assignment. Unsetting the option restores unit weights for future assignments, without relocating data.

Device identity. Weights are keyed by current bdev names. A missing or renamed configured device causes startup validation to fail until configuration is reconciled. A newly discovered device receives the matching configured weight, or one if absent. Stable identity and replacement-policy integration remain future work.

Capacity. More targets generally mean more per-target pool allocation on the selected device. If each target needs B bytes and a device serves T targets, budget at least $T \times B$ plus metadata and blobstore reserves. Equal-capacity devices with a 2:2:4 split can exhaust the four-target device while others retain free capacity. The new slot check does not check free bytes or change pool capacity accounting.

Roles. Prefer equivalent roles across compared devices. The existing load counter counts role mappings, not measured I/O or separate role-specific demand. Mixing combined-role and data-only devices is not a calibrated performance model.

Across nodes. Independently weighting each engine's SSDs does not give a faster node a proportional share of objects. Cluster-wide weighting needs a versioned placement design, agreed weights distributed with cluster metadata, fault-domain handling, and migration/rebuild compatibility. This patch leaves jump placement unchanged. [5]

6. Validation and next steps

The full SCons install build, installed policy test, all four SMD unit tests, and standalone policy test with ASan/UBSan passed for this revision. CI/package manifest checks also passed. These results do not establish a hardware speedup.

Check	Purpose
Standalone C test with ASan/UBSan	Parser rejection cases, name matching, ratio arithmetic, limits
SCons build and installed policy test	Compile the integrated engine and test executable
SMD unit tests	Real persistent-table accounting for role and system mappings
Manifest checks	Register the executable in CI and RPM server-tests

The SMD regression creates 21 combined-role targets, checks that 63 slots cannot hold another multi-role target, adds a system-target mapping, and verifies that a further addition returns overflow without inserting the new mapping. Existing mappings remain readable. The pure policy tests cover equal weights, 2:2:4 and 4:2 distributions, seeded counts, rounding, scaling invariance, and integer boundaries.

To run the standalone test from the source root:

```
cc -std=c11 -Wall -Wextra -Werror -pedantic \
  -fsanitize=address,undefined -g \
  src/bio/tests/bio_weight_test.c -o /tmp/bio_weight_test
/tmp/bio_weight_test
```

After building with tests enabled, run `install/bin/bio_weight_test` and `install/bin/smd_ut`. The BIO test group in `utils/utest.yaml` includes both. The RPM manifest includes the policy executable; an actual RPM build is still a separate validation step.

Remaining integration coverage includes unknown-name startup rejection, role filtering through the live engine, changed weights across engine restarts, partial SMD initialization, hot removal/replacement, and insufficient capacity across an entire initialization. The unit tests do not substitute for these scenarios.

For hardware evaluation, use matched fresh baseline and weighted deployments. Keep target count, object class, pool size, client concurrency, and caching fixed. Alternate repeated runs; capture medians, variability, tail latency, per-device traffic, CPU, and network use. Validate checksums and readback after restart. For dfuse-based tests, verify actual interception when using an interception library. Test sustained writes and reclamation, not only short bursts.

No throughput percentage is claimed in this document. A successful build or a policy distribution test cannot establish a raw-NVMe or multi-node performance benefit.

7. Sources and implementation map

This guide uses public DAOS documentation for the overall concepts and the checked-out source for branch-specific behavior. Older architectural descriptions can assume persistent-memory hardware; storage-mode details must be read against the deployed version.

[1] DAOS architecture (public overview): <https://docs.daos.io/v2.4/overview/architecture/>

[2] DAOS storage model (pools, containers, objects): <https://docs.daos.io/v2.6/overview/storage/>

[3] VOS design and implementation: `src/vos/README.md`, `src/vos/`

[4] BIO and SPDK integration: `src/bio/README.md`, `src/bio/bio_context.c`,
`src/bio/bio_xstream.c`

[5] Distributed placement, fault domains, and rebuild: `src/placement/JUMP_MAP.md`,
`src/placement/jump_map.c`

[6] Weight validation and local device selection: `src/bio/bio_xstream.c`:
`validate_bio_weights()`, `create_bio_bdev()`, `choose_device()`, `assign_roles()`,
`assign_xs_bdev()`

[7] Parser, comparison, and policy tests: `src/bio/bio_weight.h`,
`src/bio/tests/bio_weight_test.c`

[8] SMD table limit and regression coverage: `src/include/daos_srv/smd.h`,
`src/bio/smd/smd_device.c`, `src/bio/smd/tests/smd_ut.c`

[9] Test and package integration: `src/bio/tests/SConscript`, `utils/utest.yaml`,
`utils/rpms/daos.spec`, `ci/test_files_to_stash.txt`

Repository branch: <https://github.com/chizuchizu/daos/tree/codex/heterogeneous-nvme>

The companion PDF is `docs/admin/daos-heterogeneous-nvme.pdf`. Both documents describe generic behavior and contain no deployment-specific configuration or benchmark results.